

Same slow query. More context, **new** **answer.**

SQL only

the query text

7 / 10 runs: "add indexes" — a different set each time

+ EXPLAIN plan

the steps Postgres took

10 / 10 runs: the real culprit — a correlated subquery

+ schema & indexes

what already exists

Still an index in its top 3 — but stopped suggesting the trigram index that already existed

+ history via MCP

days of stats & plans

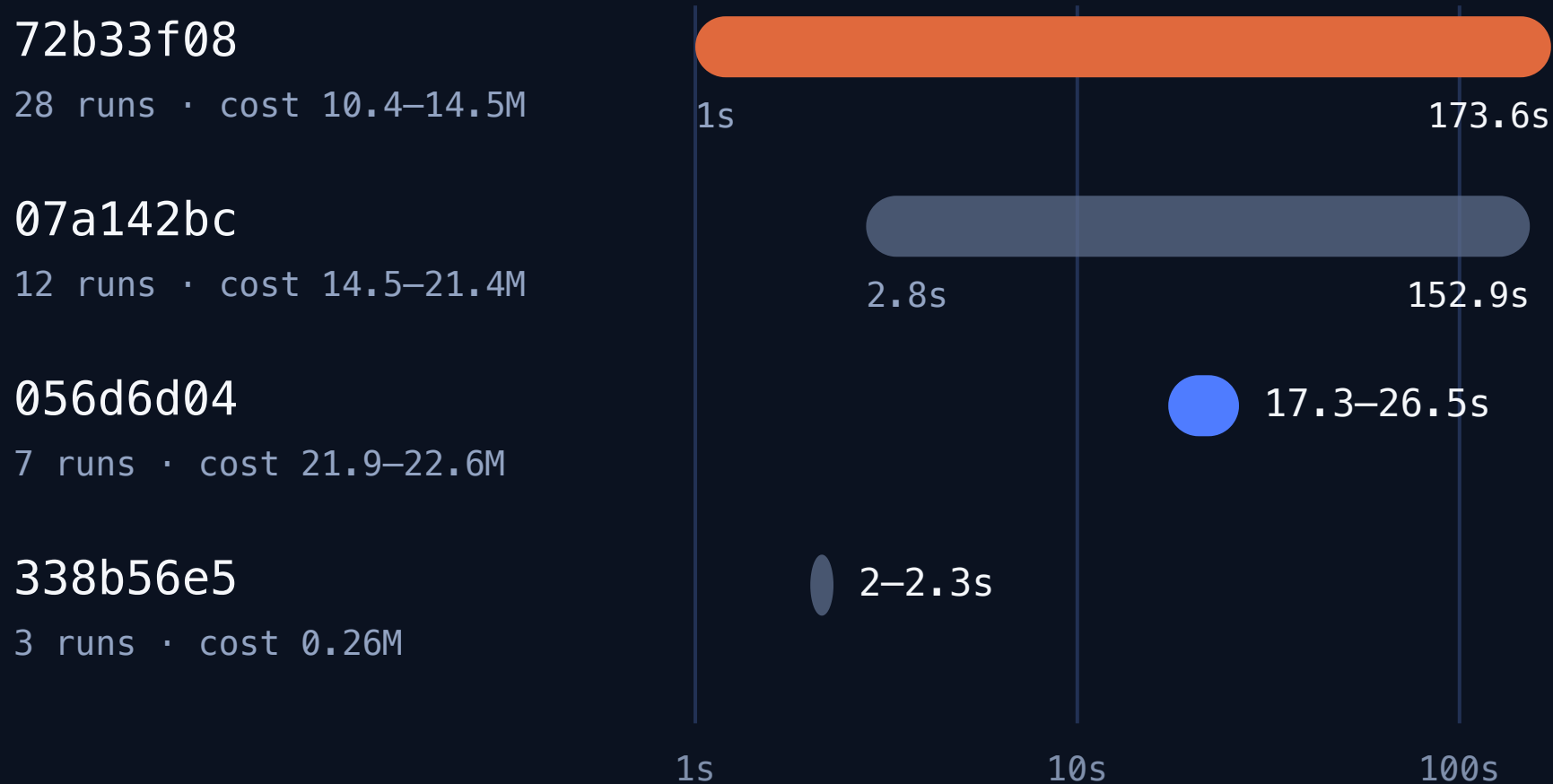
The subquery, plus a vacuum problem on one table and a missing index

Opus, Sonnet and ChatGPT all found the subquery once they had the plan. The evidence moved the answer more than the model did.

WHY ONE EXPLAIN ISN'T ENOUGH

One query. One night. Four different plans.

50 captured plans for the same SQL on the same server.
Nothing changed but the word being searched.



runtime per plan shape (log scale) · bar = fastest → slowest run

SURPRISE

The plan with the *lower* cost estimate (10.4M) hit 173.6s. The "pricier" 21.9M plan never passed 26.5s.

SO

Cost is a relative guess inside one query — not a stopwatch. Run EXPLAIN by hand and you see one of these four.

Most installs aren't set up to **remember** a slow night.

UNDER

No
pg_stat_statements,
no auto_explain,
slow-query logging
off.

Can't tell slow from
slower. Can't answer
"why" after the fact.

THE FOUR TO TURN ON

pg_stat_statements
which queries cost the most,
over time

auto_explain
saves the actual plan when a
query is slow

track_io_timing
disk wait vs CPU work (~1–5%
overhead)

log_min_duration_statement
logs slow query text · start near
1s

OVER

Log every
statement,
thresholds at 0–
100 ms, auditing
on everything.

Signal buried in
volume. The same
blindness from
the other side.

pg_stat_statements

A running total. Tells you *which*
query deserves your afternoon.

auto_explain

A plan run by hand is a
photograph. Fifty captured
overnight are a film. Only the film
shows the flip.

WHAT THE PLAN POINTED TO

98% of the time was in one subquery.

DATA PAGES READ · BEFORE (correlated subquery, re-run per row)



~193M

AFTER (materialized CTE: compute the set once, reuse it)



~1M

log scale · a page is an 8 KB block of table data, like a printed page Postgres has to read

1 Does it matter? 256 calls in 24h, already #3 by total runtime at 22%.

2 Is it always slow? No — four plan shapes, so capture the bad nights.

3 Where's the work? ~ 700 ms per loop $\times \sim 200$ rows ≈ 200 s. Fix the loop, not the index list.

→ **Next problem:** the faster query still spilled its sort to disk. That's a memory (work_mem) fix, not a query fix.

Figures as shown and spoken in the talk (Blue Box demo DB); page counts rounded. MATERIALIZED forces Postgres (12+) to compute a CTE once instead of inlining it.

THE HUMAN PART THAT STAYS

Name the problem. Then fence the agent in.

Query

The SQL asks for work it doesn't need.

fix: `rewrite`

Stats

Estimates are wrong, so the plan is wrong.

fix: `ANALYZE`, `extended stats`

Data

The work is real and the volume grew.

fix: `less data`, or `accept it`

Config

The plan is right; the memory isn't.

fix: `work_mem`, `per role`

A BOUNDED TUNING LOOP

Evidence stats + captured plans	Hypothesis "expect nested loop → hash join"	Test off-prod clone · branch · same-shape copy	Compare same parameters, baseline vs new	Stop define "done" up front
--	---	--	--	--

"AI is awesome, except it has no idea what's really happening outside of itself in your environment right now." — Ryan Booz